What makes a sketch-based long-read mapper cheaper: nine changes to shmap, by the layer they act on

Dobromir Panchev Pesho Ivanov

Abstract

Summary: shmap-rs is a Rust port of *map-shmap* [1] that is 2.14–2.97× faster single-threaded and uses 7.0–7.3× less peak memory while producing byte-identical output. This companion note accounts for where that came from. 9 changes are responsible; all 9 of them preserve output exactly, none alters a mapping decision, and each is compared against the upstream source it replaces at a pinned revision. They are presented in the order they had to be discovered: the cost structure the algorithm implies, the data structure that structure argues for, the algorithmic identities that let work be skipped, the parallel decomposition, and only last the code. That order is the result rather than the presentation: the largest wins are in the first two layers, and instruction-level work — the layer that usually attracts attention — is worth least here, and is where four further attempts measured negative.

Availability: original https://github.com/pesho-ivanov/shmap; this port https://github.com/d0bromir/shmap-rs, MPL-2.0, version 1.4.1.

1 The theoretical base

map-shmap [1] is Ivanov and Medvedev's, and nothing below changes what it computes. It sketches read and reference by FracMinHash, attributes each seed match to the overlapping fixed-width buckets whose window contains it, prunes buckets with an admissible seed heuristic, and refines the survivors with a sliding window. Its per-read cost has three factors — reads, matches examined, and a logarithmic term in reference size — and the important property for an implementer is which of them is paid in *arithmetic* and which in *memory traffic*.

Almost all of it is memory traffic: seeds are looked up in an index far larger than the last-level cache, the buckets they touch are scattered across an array sized by the reference, and the refinement sweep walks a window last touched by a different read. Profiling agrees — `map: match_rest` is the costliest stage, 21.0% of CPU on average, and pointer-chasing rather than arithmetic.

Two consequences order everything else. A change that reduces *how much memory is touched* outranks one that reduces instructions issued, because the machine is waiting either way; and the sizes that matter are those set by the *reference*, since a whole-genome run cannot escape them — which points at the accumulator.

2 The data structure

The C++ allocates one dense bucket array per reference segment, sized so that every window the algorithm could ever use has a slot (`buckets.h`, Table 1). The width is a compile-time minimum rather than the width any particular read will use, so a human genome costs about fifteen gigabytes before a single read is mapped, and that cost is paid whether one read is mapped or millions.

The observation that removes it is that *a read can only touch buckets within its own span*. The array does not have to be indexed by reference bucket; it has to be indexed by the buckets this read can reach, which is set by the read's own half-length. shmap-rs sizes it that way and keeps a sparse map as the fallback, and the two paths are tested to agree exactly. The footprint stops being a property of the genome and becomes a property of the query: 2.58–2.67 GB single-threaded against 18.85 GB.

One change, in one data structure, and it is most of the memory result. It also removed a contention bottleneck that had made multithreading nearly useless: a per-worker copy of a genome-sized array does not scale, and no thread tuning would have rescued it. The data structure had to be right before the parallel decomposition could be.

3 The algorithmic identities

Three changes skip work by proving it need not be done. None is an approximation; each is an identity, and the byte-identical output is the check.

The second-best search repeats the first. Computing mapq needs the second-best mapping, and the C++ searches for it with a second full pass over the same surviving buckets (`shmap.h`, Table 1). On the two fixed-length metrics, scoring a bucket turns out to be a pure function of its *location*: it never reads the mutable state that pruning updates between the passes, and it restores its own scratch exactly. The second pass can therefore replay the first's scores. It is deliberately not applied to the two bucket-based metrics, where the score is built out of exactly that mutable state, so the identity does not hold. Because the acceptance threshold ratchets upward during the first pass, a bucket discarded late there can still clear the second pass's flatter cutoff — so the win is partial by construction, which is why it is roughly two calls in five and not all of them.

A repetitive seed does not need a hash map. The C++ builds a fresh scratch hash map per seed to accumulate its hits; the hits are already sorted, which makes the map redundant, and the same buckets can be accumulated in place at constant integer cost per hit.

An unstable sort can reproduce a stable one. Ordering the final buckets by descending match count is the original paper's own Algorithm 4 "Optimization 2". Packing the tie-break into the sort key makes an unstable sort reproduce exactly what a stable sort would give, over keys a quarter the size of the records. Here the port is stronger than the source rather than merely faster: the C++'s `std::sort` guarantees no particular

#	Change	C++ replaced	Effect, as measured when it landed	Exact
data structure				
1	Dense bucket array sized by the read's own half-length, not the reference	`buckets.h:76-97`	~4 MB/worker vs the C++'s constant ~15 GB — the 6.9-7.4x-less-memory headline (RESULTS.md §1) is almost entirely this. On a 6,000-read whole-genome run: `bucket_merge` 1342.1 s → 39.1 s, whole run 1995.6 s → 725.6 s	yes
algorithm				
2	Streaming multi-hit seeds	`shmap.h:105-140`	Not isolated as its own standalone percentage in the record; replaces ~O(hits) hashmap inserts with O(1) integer work per hit (§2)	yes
3	`RefineCache`	`shmap.h:508-520`	44% of `find_best_mapping` calls eliminated, flat across 1x/3x/10x coverage: `match_rest_for_best2` **-66 to -67%**, `refine` **-39 to -41%**, `query_mapping` **-9 to -11%** (§3)	yes
9	Final bucket ordering: an unstable sort on a packed key reproduces stable-sort output	`buckets.h:151-174`	Sorts 8-byte packed keys instead of 32-byte records — no isolated standalone percentage in the record, but real: moving 4x fewer bytes and skipping a stable sort's temporary allocation	yes
parallelism				
4	Multithreaded read mapping	— *none*	RESULTS.md §2 (single-thread and `-@4` per benchmark) and §3 (the full thread-scaling table, including the NUMA ceiling Q4/Q5 diagnosed)	yes
5	Hash-sharded index storage + chunked reference sketching, both parallel across `-@`	— *none*	`index_initializing` **8.4 s → 1.1-1.8 s**; chunked sketching **~18% off 'indexing'** at `-@8` (§5)	yes
6	Two-pass parallel FASTA parsing	— *none*	`index_reading` **4.4 s → 1.5-1.7 s** (§6)	yes
code				
7	Sketching/index hot loop	`sketch.h:461-463`	**~13-17% off 'index_sketching'** is the bounds-check-free iteration alone (§7); the other three items in this row have no isolated standalone percentage in the record	yes
8	Lower allocation/memory traffic	`types.h:43-71`	**~11-13% off peak RSS** is the bounded read-ahead alone; **~5% wall** is `lto="fat"` alone (§8); the rest are allocation-*count* reductions (e.g. ~300 M fewer heap allocations for single-occurrence seeds) without an isolated standalone percentage in the record	yes

Table 1: Every change that makes shmap-rs cheaper than the C++, grouped by the layer it acts on, with the upstream source each replaces at commit `63f1103`. Effects are what the change was worth against the build it landed on and are deliberately not restated against the current one. They do not sum: the whole-run comparison is measured separately. *Exact* means the mapping output is byte-identical.

order among equal keys, even between its own runs, and this does.

4 The parallel decomposition

3 of the 9 exist only because the C++ is single-threaded, so they replace nothing upstream: threaded read mapping, hash-sharded index storage with chunked reference sketching, and a two-pass count-then-fill FASTA reader. The constraint that shapes all three is that output must not depend on thread count, which rules out any decomposition whose result depends on completion order. A sketch window is a pure function of the bases under it; a shard's contents are determined by the hash and not by which worker filled it; the FASTA reader's second pass writes exactly what its first counted. Determinism is checked rather than assumed, on every thread count and benchmark (15 of 15 checks).

The ceiling is again memory, not scheduling: on the four-socket machine scaling stops near 6.2×, while a single-NUMA-node machine running the identical binary continues to 12.0×. Per-node index replication was implemented five ways to remove it and net-regressed every time.

5 The code

Only two of the 9 are instruction-level, and they are last because they are worth least. The sketching hot loop precomputes its rotation tables, iterates without bounds checks, hashes once per lookup instead of twice, and sizes its output buffer from the actual distribution. The allocation work stores a single query position inline rather than in a one-element vector, has matches borrow their seeds instead of copying them, indexes the difference histogram densely, and discards the reference sequence after sketching, where the C++ keeps a second full copy of the genome for the whole run.

Their effects are in Table 1: real, but small beside the first two layers. The stronger evidence for the ordering is negative — hand-written SIMD sketching, two-bit packing, software prefetching of the pruning lookups and per-node index replication were each built or probed at full scale and each measured a loss or a wash. The sketching loop is load-port bound at six level-one loads per base, so more lanes add pressure where there is none to spare, and packing removes the wrong two of those loads. When a workload waits on memory, instruction-level effort is the layer with least left in it.

6 What each one is worth

Table 1's effects were each measured against the build that change landed on, months apart and on different inputs. They are real and deliberately not refreshed, but they share no baseline, machine or read set, and they do not sum — so the table cannot say how much of the result is which change. Figure 1 answers that separately.

Every change is switchable at run time, in one instrumented build, back to the code path it replaced. The leftmost bar has all 7 of them off; each bar to the right turns exactly one more back on, in the layer order argued above; the rightmost is the shipped build. Adjacent bars therefore differ by one change and noth-

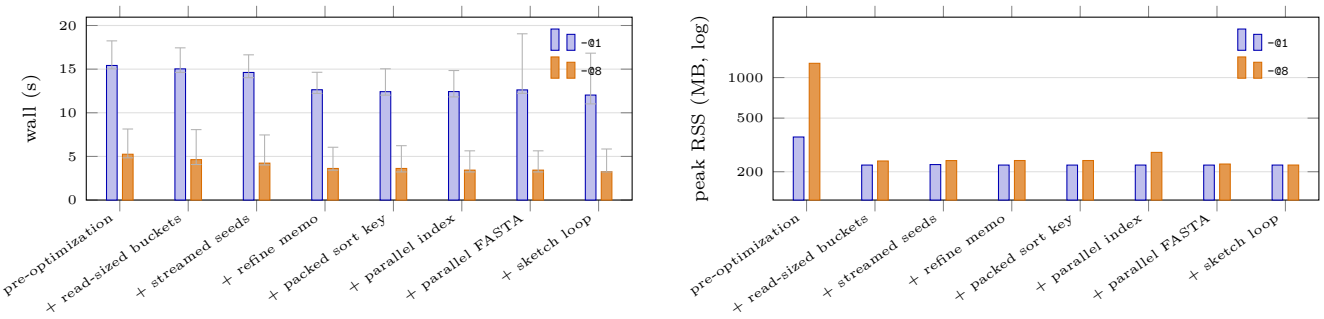

Figure 1: Every optimization, put back one at a time. Rung *pre-optimization* runs one instrumented build with every ablatable change switched off (`SHMAP_ABLATE`); each rung to its right switches exactly one more back on, so adjacent rungs differ by one change and nothing else. The two series are the two thread counts, and the gap between them is row 4; row 8 is not ablatable. Left: wall clock, median over 9 round-robin runs of the whole ladder, whiskers at min–max, so an unresolved step is visible as one. Right: peak resident set, log scale — one worker's genome-sized accumulator hides under the index-build peak, `-@8` workers' copies do not. Every rung's mapping is byte-identical; the run fails otherwise. Numbers, per-stage timers and the spread behind every bar in `figure_ablation.tsv`; the decomposition against the C++ in `reconcile.tsv`.

ing else — one binary, one machine, one compiler, one input — which is what makes a step attributable, and attributable to that change *given the ones to its left*: what the parallel index is worth depends on whether each worker still carries a genome-sized accumulator. The ladder is run 9 times round-robin, so drift cannot land on one bar and be read as that change, and every bar's mapping is compared byte for byte against the leftmost — a stricter test of the exactness claim than the suite's, because it holds the input fixed and varies only the optimization.

Those switches are not in the released mapper, deliberately: a branch and a scratch buffer that exist only to be measured are paid for by everyone who runs the program, and this note's argument is that such costs are what to be ruthless about. They live on an archived, never-merged branch, and the harness refuses a build that lacks them rather than measuring one binary 7 times and drawing a flat ladder.

2 of the 9 cannot be a rung and are reported as absent rather than estimated. Threaded read mapping is a capability rather than a branch, so it is the gap between the two series instead of a step in one; the allocation-and-layout row is type- and build-level, so reversing it would be a different binary.

End to end the ladder is 1.28× at one worker and 1.62× at 8, for 1.62× and 5.69× less peak memory; the largest single step is refine memo (row 3), worth 14.4% of the bar before it. The memory result is almost entirely one bar and is *larger* with more workers, because what it removes was per worker — the scaling argument as a measurement rather than a reason. The two parallelism rungs are flat at one worker because there they must be, both taking the serial path whichever way the switch is set, so that pair of steps is structure and not evidence. And this is chromosome scale on LAPTOP-6S4BOT55, a laptop with 8 cores, because the pre-optimization accumulator does not fit on that machine once, let alone once per worker; the figure is about the relative worth of the changes, not about how fast the program is.

Why this is not the speedup in the abstract. The ladder is worth less end to end than the 2.14–2.97× quoted there against the C++, and the two do not contradict each other: the ladder's baseline was never the C++. It is this port with the ablatable changes off, still carrying the allocation-and-layout row and every port-level difference that is not a numbered optimization at all. The two compose — by multiplication, not addition. Measured together on one input, one worker, in one sitting, the C++ over the shipped mapper is 2.22×, of which 1.75× is what the ladder cannot switch off and 1.27× is the ladder itself; the product returns the first, which makes this an identity a reader can check rather than an excuse. That total sits below the abstract's range for the reason the note gives throughout: the C++ accumulator costs a slot per five bases of *the reference*, so a chromosome flatters it and a genome does not.

7 What none of them changed

All 9 preserve output exactly, enforced rather than claimed: the mapping is byte-identical across every thread count and against the C++, ground-truth accuracy is unchanged at 98.76–99.21%, and any drop in mapped or confident reads blocks a merge. Table 1 is generated from the repository's own account of what changed, each row citing upstream at `63f1103`, so this note cannot drift from the code.

References

[1] Ivanov,P. and Medvedev,P. map-shmap: practical long-read mapping with a seed heuristic on sketches. Manuscript in preparation.